МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01. «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №1
По дисциплине: Теория алгоритмов
На тему: «Реализация двумерного клеточного автомата
с окрестностью фон Неймана»
Вариант 43

Выполнил студент
группы 5130201/40101 _____ Храмов В.Д.

Принял преподаватель _____ Востров А.В.

«____» _____ 2026 г.

Санкт-Петербург
2026

Содержание

Введение **3**

1 Математическое описание **4**

1.1 Конечный автомат 4

1.2 Клеточный автомат 4

1.3 Окрестность фон Неймана 5

1.4 Функция перехода и номер правила 5

1.5 Граничные условия 7

1.6 Синхронное обновление 9

1.7 Сходимость и периодичность 9

1.8 Классификация Вольфрама 9

2 Особенности реализации **11**

2.1 Класс CellularAutomaton 11

2.2 Класс Main 19

3 Результаты работы программы **26**

4 Анализ клеточного автомата **33**

4.1 Свойства правила 33

4.2 Простые паттерны 34

4.3 Случайные начальные условия 39

4.4 Периодичность 41

4.5 Классификация по Вольфраму 43

Заключение **44**

Список литературы **46**

Приложение. Исходный код программы **47**

Введение

В отчете представлено описание выполнения лабораторной работы №1 по дисциплине «Теория алгоритмов».

Необходимо реализовать двумерный клеточный автомат с окрестностью фон Неймана в соответствии с полученным номером №. Граничные условия — торроидальные. Пользователь определяет ширину поля и количество итераций. Необходимо учесть возможность ввода различных начальных условий (как вручную, так и случайным образом) по выбору пользователя. Реализация возможна в консоли или в графике. В итоге необходимо проанализировать свой клеточный автомат в отчете (его поведение, паттерны, «сходимость», периодичность, классификация по Вольфраму и т.д.). № = номер варианта $\cdot$ 11 $\cdot$ год рождения $\cdot$ месяц $\cdot$ день.

Для варианта 43 номер правила равен $№ = 43 \cdot 11 \cdot 2006 \cdot 11 \cdot 2 = 20\,874\,436$.

Программа реализована на языке Java в виде консольного приложения.

1 Математическое описание

1.1 Конечный автомат

Конечным автоматом Мура называется набор

$$M = (Q, X, Y, \delta, \lambda, q_0),$$

где

- Q — конечное множество состояний;
- X — конечный входной алфавит;
- Y — конечный выходной алфавит;
- $\delta : Q \times X \to Q$ — функция переходов;
- $\lambda : Q \to Y$ — функция выходов;
- $q_0 \in Q$ — начальное состояние.

1.2 Клеточный автомат

Клеточный автомат — это совокупность одинаковых конечных автоматов (клеток), расположенных в узлах регулярной решетки и соединенных между собой так, что входной символ каждого автомата образуют выходные символы его соседей. Все автоматы работают синхронно, по общим тактам времени [1, 2].

Формально клеточный автомат задается тройкой

$$A = (L, \mathcal{N}, M),$$

где

- $L = \mathbb{Z}_n \times \mathbb{Z}_n$ — решетка клеток: квадратное поле $n \times n$ с тороидальными граничными условиями;
- $\mathcal{N} = \big((-1,0),\ (0,1),\ (0,-1),\ (1,0)\big)$ — окрестность фон Неймана: сдвиги к соседям N, E, W, S; вход клетки (i,j) соединен с выходами клеток $(i,j)+v$, $v \in \mathcal{N}$;
- $M = (Q, X, Y, \delta, \lambda)$ — автомат Мура клетки без фиксированного начального состояния: $Q = Y = \{0,1\}$ (0 — «мертвая», 1 — «жи-

вая»), $\lambda(q) = q$, $X = Q^4$, $\delta\big(C, (N, E, W, S)\big) = f(C, N, E, W, S)$, где $f : Q^5 \to Q$ задается номером правила.

Отображение $\sigma^t : L \to Q$, сопоставляющее каждой клетке ее состояние в момент t, называется конфигурацией. Начальная конфигурация σ^0 задается пользователем: клетка (i, j) — это автомат M с начальным состоянием $q_0 = \sigma^0(i, j)$.

1.3 Окрестность фон Неймана

Окрестность фон Неймана клетки (i, j) образуют сама клетка C и четыре клетки, имеющие с ней общую сторону: северная N, восточная E, южная S и западная W (рис. 1):

$$C = (i, j), \quad N = (i-1, j), \quad E = (i, j+1), \quad S = (i+1, j), \quad W = (i, j-1),$$

где индексы берутся по модулю n. Строки нумеруются сверху вниз, поэтому северный сосед находится в строке с меньшим номером.

Четыре соседа N, E, W, S образуют вход автомата клетки, а сама клетка C участвует в переходе через собственное состояние. Далее буквами C, N, E, W, S обозначаются также состояния этих клеток в момент t, например $C = \sigma^t(i, j)$, $N = \sigma^t(i-1, j)$.

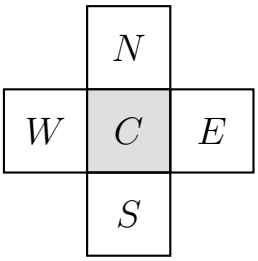


Рис. 1: Окрестность фон Неймана

1.4 Функция перехода и номер правила

Окрестность состоит из 5 клеток, каждая из которых имеет 2 состояния, поэтому существует $2^5 = 32$ различных набора (C, N, E, W, S). Функция перехода $f : Q^5 \to Q$ сопоставляет каждому набору новое состояние

центральной клетки:

$$\sigma^{t+1}(i,j) = f(C,N,E,W,S),$$

то есть полностью задается 32 значениями. Всего существует $2^{32} \approx 4{,}3 \cdot 10^9$ таких функций.

Пронумеруем наборы числами $idx \in \{0,\ldots,31\}$, двоичная запись которых есть $C\,N\,E\,W\,S$:

$$idx = 16C + 8N + 4E + 2W + S.$$

Номер правила — это число, разряд номер idx двоичной записи которого равен значению f на наборе с номером idx:

$$№ = \sum_{idx=0}^{31} f(C,N,E,W,S) \cdot 2^{idx},$$

откуда

$$f(C,N,E,W,S) = \left\lfloor \frac{№}{2^{idx}} \right\rfloor \bmod 2.$$

Номер правила в двоичной системе счисления (дополнен незначащими нулями до 32 разрядов):

$$20\,874\,436_{10} = 0000\,0001\,0011\,1110\,1000\,0100\,1100\,0100_2.$$

Таблица истинности функции f приведена в таблице 1.

Таблица 1: Таблица истинности функции перехода (правило 20 874 436, порядок $C\,N\,E\,W\,S$)

idx	C	N	E	W	S	f
0	0	0	0	0	0	0
1	0	0	0	0	1	0
2	0	0	0	1	0	1
3	0	0	0	1	1	0
4	0	0	1	0	0	0
5	0	0	1	0	1	0
6	0	0	1	1	0	1
7	0	0	1	1	1	1
8	0	1	0	0	0	0
9	0	1	0	0	1	0
10	0	1	0	1	0	1
11	0	1	0	1	1	0
12	0	1	1	0	0	0
13	0	1	1	0	1	0
14	0	1	1	1	0	0
15	0	1	1	1	1	1

idx	C	N	E	W	S	f
16	1	0	0	0	0	0
17	1	0	0	0	1	1
18	1	0	0	1	0	1
19	1	0	0	1	1	1
20	1	0	1	0	0	1
21	1	0	1	0	1	1
22	1	0	1	1	0	0
23	1	0	1	1	1	0
24	1	1	0	0	0	1
25	1	1	0	0	1	0
26	1	1	0	1	0	0
27	1	1	0	1	1	0
28	1	1	1	0	0	0
29	1	1	1	0	1	0
30	1	1	1	1	0	0
31	1	1	1	1	1	0

По таблице видно, что

- живая клетка выживает при 6 конфигурациях соседей из 16, мертвая рождается при 5 из 16;
- пустая окрестность остается пустой ($idx = 0 \to 0$), а клетка, у которой живы все 5 клеток окрестности, умирает ($idx = 31 \to 0$);
- новое состояние зависит не только от состояния клетки и количества живых соседей, но и от того, какие именно соседи живы. Например, мертвая клетка с одним живым западным соседом рождается ($idx = 2 \to 1$), а с одним живым восточным — нет ($idx = 4 \to 0$).

1.5 Граничные условия

В работе используются тороидальные граничные условия: противоположные края поля считаются соседними. Правый край поля склеивается с левым, а верхний — с нижним, и поле превращается в поверхность тора, у которой нет границ. Все клетки при этом оказываются равноправными: окрестность любой клетки устроена одинаково.

Формально индексы клеток являются элементами $\mathbb{Z}_n$, а сложение индексов выполняется по модулю n. Соседи клетки (i, j):

$$N = \big((i-1) \bmod n,\ j\big), \qquad E = \big(i,\ (j+1) \bmod n\big),$$
$$S = \big((i+1) \bmod n,\ j\big), \qquad W = \big(i,\ (j-1) \bmod n\big),$$

где $a \bmod n \in \{0, \ldots, n-1\}$ — остаток от деления a на n; в частности, $(-1) \bmod n = n-1$ и $n \bmod n = 0$.

Для внутренних клеток ($0 < i < n-1$, $0 < j < n-1$) приведение по модулю ничего не меняет. Для клеток на краях поля один сосед берется с противоположного края:

- верхняя строка ($i = 0$): северный сосед — клетка $(n-1, j)$;
- нижняя строка ($i = n-1$): южный сосед — клетка $(0, j)$;
- левый столбец ($j = 0$): западный сосед — клетка $(i, n-1)$;
- правый столбец ($j = n-1$): восточный сосед — клетка $(i, 0)$.

Угловая клетка лежит сразу на двух краях, поэтому с противоположных краев берутся два ее соседа: один по вертикали и один по горизонтали (таблица 2).

Таблица 2: Соседи угловых клеток на торе

Клетка	N	E	W	S
$(0, 0)$	$(n-1, 0)$	$(0, 1)$	$(0, n-1)$	$(1, 0)$
$(0, n-1)$	$(n-1, n-1)$	$(0, 0)$	$(0, n-2)$	$(1, n-1)$
$(n-1, 0)$	$(n-2, 0)$	$(n-1, 1)$	$(n-1, n-1)$	$(0, 0)$
$(n-1, n-1)$	$(n-2, n-1)$	$(n-1, 0)$	$(n-1, n-2)$	$(0, n-1)$

На торе четыре угловые клетки поля образуют квадрат 2×2: каждая из них соседствует с двумя другими угловыми клетками, а с противоположной по диагонали — нет, так как диагональные клетки в окрестность фон Неймана не входят.

Далее считается, что $n \geq 3$: тогда все четыре соседа любой клетки различны и отличны от нее самой. При $n = 2$ северный и южный соседи совпадали бы, так как $(i-1) \bmod 2 = (i+1) \bmod 2$.

1.6 Синхронное обновление

Все клетки автомата обновляются одновременно: новое состояние каждой клетки вычисляется только по конфигурации σ^t:

$$C_{i,j}^{t+1} = f\left(C_{i,j}^t,\ C_{i-1,j}^t,\ C_{i,j+1}^t,\ C_{i,j-1}^t,\ C_{i+1,j}^t\right).$$

Если записывать новые значения в то же поле, из которого читаются соседи, то часть клеток увидит соседей уже из поколения $t{+}1$, и результат будет зависеть от порядка обхода поля. Поэтому для вычисления шага используются два поля: из одного читается поколение t, в другое записывается поколение $t+1$.

1.7 Сходимость и периодичность

Поле $n \times n$ с двумя состояниями клетки имеет конечное число конфигураций — 2^{n^2}. Автомат детерминирован: из одной конфигурации всегда получается одна и та же. Поэтому не более чем через 2^{n^2} шагов некоторая конфигурация обязательно повторится, и начиная с этого момента эволюция становится периодической:

$$\sigma^0 \to \sigma^1 \to \cdots \to \sigma^{t_0} \to \cdots \to \sigma^{t_0+p-1} \to \sigma^{t_0+p} = \sigma^{t_0}.$$

Здесь t_0 — предпериод (длина переходного процесса), p — период. Если $p = 1$, конфигурация больше не меняется — это неподвижная точка. Если $\sigma^j = \sigma^i$ для некоторых $i < j$ и все предыдущие конфигурации различны, то $t_0 = i$, $p = j - i$.

1.8 Классификация Вольфрама

С. Вольфрам предложил разделить клеточные автоматы на четыре класса по характеру их эволюции из случайных начальных конфигураций [3]:

1. Класс I — эволюция приводит к однородному состоянию (все клетки мертвые или все живые).

2. Класс II — эволюция приводит к набору простых устойчивых или периодических структур, разделенных друг от друга.
3. Класс III — хаотическое (псевдослучайное) поведение. Конфигурации не упорядочиваются, малые изменения начальных условий распространяются по полю.
4. Класс IV — сложные локализованные структуры, которые движутся и взаимодействуют между собой; поведение на границе порядка и хаоса.

2 Особенности реализации

Программа написана на языке Java и состоит из двух классов:

- `CellularAutomaton` — логика автомата: хранение поля, правило перехода, шаг эволюции, начальные условия, обнаружение цикла;
- `Main` — консольный интерфейс: ввод параметров с проверкой, выбор начальных условий, вывод поля и итогов.

Полный исходный код приведен в Приложении.

2.1 Класс CellularAutomaton

Класс содержит следующие поля:

- `long rule` — номер правила;
- `int size` — ширина квадратного поля;
- `boolean[][] cells` — текущее поколение;
- `boolean[][] next` — буфер для следующего поколения;
- `int generation` — номер текущего поколения;
- `Map<BitSet, Integer> history` — все встреченные конфигурации и номера поколений, в которых они впервые появились;
- `boolean cycleFound`, `int transientLength`, `int period` — признак найденного цикла, предпериод и период.

Конструктор CellularAutomaton()

Конструктор вычисляет номер правила по формуле из задания и создает два пустых поля заданного размера. Приведение первого множителя к `long` гарантирует, что все произведение вычисляется в 64-битной арифметике без переполнения. Код представлен в листинге 1.

Входные данные: `int numberOption, constValue, year, month, day` — номер варианта, константа 11 и дата рождения из формулы номера правила; `int size` — ширина поля.

Выходные данные: созданный объект автомата с вычисленным номером правила и пустым полем (все клетки мертвые, номер поколения 0).

```
1 public CellularAutomaton(int numberOption, int constValue, int year, int
    month, int day, int size) {
2     this.rule = (long) numberOption * constValue * year * month * day;
```

```
    this.size = size;
    this.cells = new boolean[this.size][this.size];
    this.next = new boolean[this.size][this.size];
}
```

Листинг 1: Конструктор класса CellularAutomaton

Метод bit()

Метод `bit` возвращает k-й бит числа x: число сдвигается вправо на k разрядов, после чего побитовым «И» с единицей выделяется младший бит. Код представлен в листинге 2.

Входные данные: `long x` — число, `int k` — номер бита.

Выходные данные: значение бита, 0 или 1.

```
static int bit(long x, int k) {
    long shifted = x >> k;
    long lastBit = shifted & 1;
    return (int) lastBit;
}
```

Листинг 2: Метод bit()

Метод newState()

Метод `newState` применяет `bit` к номеру правила, то есть вычисляет значение функции перехода f на наборе с номером `idx`. Код представлен в листинге 3.

Входные данные: `int idx` — индекс окрестности (0..31).

Выходные данные: новое состояние клетки, 0 или 1.

```
int newState(int idx) {
    return bit(rule, idx);
}
```

Листинг 3: Метод newState()

Метод wrap()

Метод `wrap` реализует тороидальные граничные условия: переводит любую координату в диапазон $0..size-1$. Код представлен в листинге 4.

Входные данные: `int coord` — координата (строка или столбец), возможно, вышедшая за край поля.

Выходные данные: координата $coord \bmod size$ в диапазоне $0..size-1$.

```
private int wrap(int coord) {
    return (coord % size + size) % size;
}
```

Листинг 4: Метод wrap()

Метод getWrapped()

Метод `getWrapped` возвращает состояние клетки, приводя обе координаты методом `wrap`. Код представлен в листинге 5.

Входные данные: `int row, col` — координаты клетки, возможно, вышедшие за край поля.

Выходные данные: состояние клетки `cells[wrap(row)][wrap(col)]` (`true` — живая).

```
private boolean getWrapped(int row, int col) {
    return cells[wrap(row)][wrap(col)];
}
```

Листинг 5: Метод getWrapped()

Метод neighborhoodIndex()

Метод собирает индекс окрестности фон Неймана клетки (row, col) в порядке $C\,N\,E\,W\,S$. Состояние каждой из пяти клеток читается с учетом тора и переводится в 0 или 1 методом `toBit`, после чего биты складываются с весами 16, 8, 4, 2, 1. Код представлен в листинге 6.

Входные данные: `int row, col` — координаты клетки.

Выходные данные: индекс окрестности $idx = 16C+8N+4E+2W+S$ в диапазоне 0..31.

```
int neighborhoodIndex(int row, int col) {
    int n = toBit(getWrapped(row - 1, col));
    int c = toBit(getWrapped(row, col));
    int s = toBit(getWrapped(row + 1, col));
    int w = toBit(getWrapped(row, col - 1));
    int e = toBit(getWrapped(row, col + 1));
    return c * 16 + n * 8 + e * 4 + w * 2 + s;
```

```
}
```

Листинг 6: Метод neighborhoodIndex()

Метод toBit()

Вспомогательный метод `toBit` переводит `boolean` в число: `true` → 1, `false` → 0. Код представлен в листинге 7.

Входные данные: `boolean value` — состояние клетки.

Выходные данные: 1 или 0.

```
private static int toBit(boolean value) {
    return value ? 1 : 0;
}
```

Листинг 7: Метод toBit()

Метод step()

Метод выполняет один шаг эволюции. Для каждой клетки вычисляется индекс окрестности по текущему полю `cells`, новое состояние записывается в буфер `next`. После обработки всех клеток ссылки на массивы меняются местами: новое поколение становится текущим, а старый массив используется как буфер на следующем шаге. Копирования массивов при этом не происходит. Перед первым шагом в историю записывается начальная конфигурация, а после каждого шага выполняется проверка на повторение конфигурации. Сложность одного шага — $O(n^2)$. Код представлен в листинге 8.

Входные данные: используется текущее поле `cells`.

Выходные данные: поле `cells` заменяется следующим поколением, номер поколения `generation` увеличивается на 1, при повторении конфигурации заполняются поля `cycleFound`, `transientLength`, `period`.

```
public void step() {
    if (history.isEmpty()) {
        history.put(snapshot(), generation);
    }

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            int idx = neighborhoodIndex(i, j);
```

```
            next[i][j] = newState(idx) == 1;
        }
    }

    boolean[][] tmp = cells;
    cells = next;
    next = tmp;

    generation++;
    checkCycle();
}
```

Листинг 8: Метод step()

Метод snapshot()

Метод `snapshot` упаковывает поле в `BitSet`: клетка (row, col) соответствует биту $row \cdot size + col$, т. е. на клетку тратится один бит. Методы `equals` и `hashCode` класса `BitSet` сравнивают содержимое, поэтому его можно использовать как ключ хеш-таблицы, в отличие от массива `boolean[][]`. Код представлен в листинге 9.

Входные данные: используется текущее поле `cells`.

Выходные данные: `BitSet` из $size^2$ битов, в котором установлены биты живых клеток.

```
private BitSet snapshot() {
    BitSet bits = new BitSet(size * size);
    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            if (cells[row][col]) {
                bits.set(row * size + col);
            }
        }
    }
    return bits;
}
```

Листинг 9: Метод snapshot()

Метод checkCycle()

Для обнаружения цикла все встреченные конфигурации хранятся в хеш-таблице `history` (конфигурация → номер поколения). Метод `checkCycle`

пытается добавить текущую конфигурацию в таблицу методом `putIfAbsent`. Если такая конфигурация уже встречалась в поколении $prev$, метод возвращает $prev$, и тогда предпериод равен $prev$, а период — $generation - prev$. После обнаружения цикла конфигурации больше не сохраняются. Сравнение конфигураций точное (без риска коллизий), а затраты памяти составляют $O(T \cdot n^2/8)$ байт, где T — число выполненных шагов. Код представлен в листинге 10.

Входные данные: используются текущее поле, номер поколения `generation` и таблица `history`.

Выходные данные: текущая конфигурация добавляется в `history`, предпериод `transientLength` и период `period`.

```
private void checkCycle() {
    if (cycleFound) {
        return;
    }
    Integer prev = history.putIfAbsent(snapshot(), generation);
    if (prev != null) {
        cycleFound = true;
        transientLength = prev;
        period = generation - prev;
    }
}
```

Листинг 10: Метод checkCycle()

Метод randomize()

Метод заполняет поле случайным образом: каждая клетка становится живой с вероятностью `density`, независимо от остальных. Для этого используется равномерно распределенное число из $[0, 1)$, возвращаемое `Random.nextDouble()`: оно меньше `density` ровно с вероятностью `density`. Ожидаемое число живых клеток равно $density \cdot n^2$. Генератор псевдослучайных чисел инициализируется заданным значением `seed`, поэтому одинаковый `seed` всегда дает одинаковое поле, что позволяет воспроизводить эксперименты. Код представлен в листинге 11.

Входные данные: `double density` — плотность живых клеток (0..1), `long seed` — начальное значение генератора.

Выходные данные: поле `cells` заполняется случайно.

```java
public void randomize(double density, long seed) {
    Random random = new Random(seed);
    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            cells[row][col] = random.nextDouble() < density;
        }
    }
}
```

Листинг 11: Метод randomize()

Метод getCell()

Метод возвращает состояние клетки. Код представлен в листинге 12.

Входные данные: `int row, col` — координаты клетки в диапазоне $0..size-1$.

Выходные данные: `true`, если клетка живая.

```java
public boolean getCell(int row, int col) {
    return cells[row][col];
}
```

Листинг 12: Метод getCell()

Метод setCell()

Метод задает состояние клетки, используется при ручном вводе и при размещении клетки в центре поля. Код представлен в листинге 13.

Входные данные: `int row, col` — координаты клетки, `boolean value` — новое состояние.

Выходные данные: изменяется клетка поля.

```java
public void setCell(int row, int col, boolean value) {
    cells[row][col] = value;
}
```

Листинг 13: Метод setCell()

Метод clear()

Метод делает все клетки поля мертвыми. Код представлен в листинге 14.

Входные данные: текущее поле `cells`.

Выходные данные: поле очищено.

```
public void clear() {
    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            cells[row][col] = false;
        }
    }
}
```

Листинг 14: Метод clear()

Метод population()

Метод подсчитывает число живых клеток поля. Код представлен в листинге 15.

Входные данные: текущее поле `cells`.

Выходные данные: число живых клеток от 0 до $size^2$.

```
public int population() {
    int count = 0;
    for (int row = 0; row < size; row++) {
        for (int col = 0; col < size; col++) {
            if (cells[row][col]) {
                count++;
            }
        }
    }
    return count;
}
```

Листинг 15: Метод population()

Методы доступа

Методы доступа возвращают значения закрытых полей класса (листинги 16–20):

- `getSize` — ширина поля `size`;
- `getGeneration` — номер текущего поколения `generation`;
- `isCycleFound` — признак найденного цикла `cycleFound`;
- `getTransientLength` — предпериод `transientLength` (0, если цикл не найден);

- `getPeriod` — период `period` (0, если цикл не найден).

```
public int getSize() {
    return size;
}
```

Листинг 16: Метод getSize()

```
public int getGeneration() {
    return generation;
}
```

Листинг 17: Метод getGeneration()

```
public boolean isCycleFound() {
    return cycleFound;
}
```

Листинг 18: Метод isCycleFound()

```
public int getTransientLength() {
    return transientLength;
}
```

Листинг 19: Метод getTransientLength()

```
public int getPeriod() {
    return period;
}
```

Листинг 20: Метод getPeriod()

2.2 Класс Main

Класс `Main` реализует консольный интерфейс программы.

Метод main()

Метод запрашивает у пользователя ширину поля (3..100), количество итераций (1..100 000), начальные условия, шаг вывода k (печатается каждое k-е поколение) и необходимость остановки при обнаружении цикла. Затем выполняется заданное число итераций; печатаются начальное поле, каждое k-е поколение, последнее поколение и поколение, на котором обнаружен цикл. В конце выводится итог работы. Код представлен в листинге 21.

Входные данные: значения вводятся пользователем.

Выходные данные: поколения и итог работы выводятся в консоль.

Минимальная ширина поля равна 3, так как при $n \leq 2$ на торе северный и южный (западный и восточный) соседи клетки совпадают, и окрестность фон Неймана вырождается.

```
printGeneration(automaton);
for (int i = 1; i <= iterations; i++) {
    boolean hadCycle = automaton.isCycleFound();
    automaton.step();
    boolean cycleJustFound = !hadCycle && automaton.isCycleFound();

    if (i % printEvery == 0 || i == iterations || cycleJustFound) {
        printGeneration(automaton);
    }
    if (cycleJustFound && stopOnCycle) {
        break;
    }
}

printSummary(automaton);
```

Листинг 21: Основной цикл метода main()

Метод readInt()

Метод `readInt` запрашивает целое число до тех пор, пока не будет введено корректное значение из диапазона $[min, max]$. При пустом вводе возвращается значение по умолчанию, которое показывается в подсказке в квадратных скобках. Ввод читается целой строкой и преобразуется в число методом `Integer.parseInt`; некорректный ввод перехватывается через исключение `NumberFormatException`. Код представлен в листинге 22.

Входные данные: `String prompt` — подсказка, `int min, max` — границы диапазона, `int def` — значение по умолчанию.

Выходные данные: введенное число из $[min, max]$ или `def`.

```
static int readInt(String prompt, int min, int max, int def) {
    while (true) {
        String s = readLine(prompt + " [" + def + "]: ");
        if (s.isEmpty()) {
            return def;
        }
        try {
            int value = Integer.parseInt(s);
```

```
            if (value >= min && value <= max) {
                return value;
            }
        } catch (NumberFormatException ignored) {
        }
        println("  Введите целое число от " + min + " до " + max + ".");
    }
}
```

Листинг 22: Метод readInt()

Метод readLine()

Метод **readLine** выводит подсказку, читает строку и удаляет пробелы по краям. При окончании ввода (EOF) метод **readln** возвращает **null**, и тогда возвращается пустая строка. Код представлен в листинге 23.

Входные данные: **String prompt** — подсказка.

Выходные данные: введенная строка без пробелов по краям или пустая строка.

```
static String readLine(String prompt) {
    String s = readln(prompt);
    return s == null ? "" : s.trim();
}
```

Листинг 23: Метод readLine()

Метод initField()

Метод выводит меню начальных условий и заполняет поле выбранным способом (листинг 24):

1. случайно — запрашиваются плотность живых клеток в процентах и **seed**, после чего вызывается метод **randomize**;
2. вручную — метод **readManual**;
3. одна живая клетка в центре поля, клетка $(\lfloor size/2 \rfloor, \lfloor size/2 \rfloor)$;
4. пустое поле.

Входные данные: **CellularAutomaton automaton** — автомат, поле которого заполняется; номер способа вводится пользователем.

Выходные данные: поле автомата заполнено выбранным способом.

```
static void initField(CellularAutomaton automaton) {
```

```
    println();
    println("Начальные условия:");
    println("  1 - случайно");
    println("  2 - вручную");
    println("  3 - одна клетка в центре");
    println("  4 - пустое поле");
    int mode = readInt("Выбор", 1, 4, 1);

    switch (mode) {
        case 1 -> {
            int percent = readInt("Плотность живых клеток, % (0...100)",
                0, 100, 50);
            long seed = readSeed();
            automaton.randomize(percent / 100.0, seed);
        }
        case 2 -> readManual(automaton);
        case 3 -> {
            int center = automaton.getSize() / 2;
            automaton.setCell(center, center, true);
        }
        default -> automaton.clear();
    }
}
```

Листинг 24: Метод initField()

Метод readSeed()

Метод запрашивает начальное значение генератора случайных чисел до тех пор, пока не будет введено целое число или пустая строка. Если `seed` не введен, он берется из текущего времени (`System.nanoTime()`) и печатается, чтобы эксперимент можно было повторить. Код представлен в листинге 25.

Входные данные: значение вводится пользователем.

Выходные данные: `long` — введенный или сгенерированный `seed`.

```
static long readSeed() {
    while (true) {
        String s = readLine("Seed (пусто = случайный): ");
        if (s.isEmpty()) {
            long seed = System.nanoTime();
            println("Используется seed = " + seed);
            return seed;
        }
        try {
```

```
            return Long.parseLong(s);
        } catch (NumberFormatException e) {
            println("  Введите целое число или оставьте пустым.");
        }
    }
}
```

Листинг 25: Метод readSeed()

Метод readManual()

Метод реализует ручной ввод начальной конфигурации: пользователь вводит строки поля сверху вниз, символы `+` и `1` обозначают живую клетку, любые другие — мертвую. Строка короче ширины поля дополняется мертвыми клетками, лишние символы игнорируются, пустая строка завершает ввод (оставшиеся строки — мертвые). Код представлен в листинге 26.

Входные данные: `CellularAutomaton automaton` — автомат, поле которого заполняется; строки поля вводятся пользователем.

Выходные данные: поле автомата заполнено по вводу пользователя.

```
static void readManual(CellularAutomaton automaton) {
    int size = automaton.getSize();
    println("Вводите строки поля сверху вниз (до " + size + " строк).");
    println("Живая клетка: + или 1, мёртвая: . или 0. Пустая строка - зако
       нчить ввод.");
    automaton.clear();
    for (int row = 0; row < size; row++) {
        String line = readln(String.format("%3d> ", row));
        if (line == null || line.isBlank()) {
            break;
        }
        for (int col = 0; col < Math.min(line.length(), size); col++) {
            char ch = line.charAt(col);
            automaton.setCell(row, col, ch == '+' || ch == '1');
        }
    }
}
```

Листинг 26: Метод readManual()

Метод printField()

Метод печатает поле построчно: + — живая клетка, пробел — мертвая. Код представлен в листинге 27.

Входные данные: `CellularAutomaton automaton` — автомат.

Выходные данные: поле выводится в консоль.

```
static void printField(CellularAutomaton automaton) {
    for (int row = 0; row < automaton.getSize(); row++) {
        for (int col = 0; col < automaton.getSize(); col++) {
            print(automaton.getCell(row, col) ? "+ " : "  ");
        }
        println();
    }
    println();
}
```

Листинг 27: Метод printField()

Метод printGeneration()

Метод печатает заголовок с номером поколения, числом живых клеток и, если цикл уже найден, предпериодом и периодом, а затем само поле методом `printField`. Код представлен в листинге 28.

Входные данные: `CellularAutomaton automaton` — автомат.

Выходные данные: заголовок и поле выводятся в консоль.

```
static void printGeneration(CellularAutomaton automaton) {
    String header = "Поколение " + automaton.getGeneration() + " | живых:
        " + automaton.population();
    if (automaton.isCycleFound()) {
        header += " | цикл: предпериод = " + automaton.getTransientLength
            () + ", период = " + automaton.getPeriod();
    }
    println(header);
    printField(automaton);
}
```

Листинг 28: Метод printGeneration()

Метод printSummary()

Метод выводит итог: число выполненных итераций, число живых клеток, а также поколение, на котором найден цикл, предпериод и период

либо сообщение о том, что цикл не обнаружен. Код представлен в листинге 29.

Входные данные: `CellularAutomaton automaton` — автомат.

Выходные данные: итог выводится в консоль.

```
static void printSummary(CellularAutomaton automaton) {
    int size = automaton.getSize();
    println("Итог");
    println("Выполнено итераций: " + automaton.getGeneration());
    println("Живых клеток: " + automaton.population() + " из " + size *
        size);
    if (automaton.isCycleFound()) {
        int t = automaton.getTransientLength();
        int p = automaton.getPeriod();
        println("Цикл найден на поколении " + (t + p));
        println("Предпериод: " + t);
        println("Период: " + p + (p == 1 ? " (неподвижная точка)" : ""));
    } else {
        println("Цикл за " + automaton.getGeneration() + " итераций не обн
            аружен.");
    }
}
```

Листинг 29: Метод printSummary()

3 Результаты работы программы

Ниже приведены примеры работы программы (вывод в консоль вместе с введенными пользователем значениями). Программа запускается командой `./gradlew run` из каталога проекта. Поле выводится построчно: символ `+` обозначает живую клетку, пробел — мертвую.

После запуска программа выводит заголовок, запрашивает ширину поля и количество итераций, а затем предлагает выбрать способ задания начальных условий (рис. 2). В квадратных скобках указаны значения по умолчанию, которые принимаются при пустом вводе.

```
Двумерный клеточный автомат

Ширина поля (3...100) [20]: 20
Количество итераций (1...100000) [10]: 1

Начальные условия:
  1 - случайно
  2 - вручную
  3 - одна клетка в центре
  4 - пустое поле
Выбор [1]: 1
```

Рис. 2: Запуск программы и ввод параметров

Все значения вводятся с проверкой: если введено не целое число (`abc`, `2.5`) или число вне допустимого диапазона (`-5`, `1000`), программа выводит сообщение с допустимым диапазоном и повторяет запрос. Для `seed` допускается любое целое число или пустой ввод. На рис. 3 показан пример запуска, в котором для каждого параметра сначала было введено некорректное значение, а затем корректное. После ввода всех параметров программа продолжает работу в обычном режиме.

```
Двумерный клеточный автомат

Ширина поля (3...100) [20]: abc
  Введите целое число от 3 до 100.
Ширина поля (3...100) [20]: -5
  Введите целое число от 3 до 100.
Ширина поля (3...100) [20]: 1000
  Введите целое число от 3 до 100.
Ширина поля (3...100) [20]: 5
Количество итераций (1...100000) [10]: 2.5
  Введите целое число от 1 до 100000.
Количество итераций (1...100000) [10]: 0
  Введите целое число от 1 до 100000.
Количество итераций (1...100000) [10]: 3

Начальные условия:
  1 - случайно
  2 - вручную
  3 - одна клетка в центре
  4 - пустое поле
Выбор [1]: 7
  Введите целое число от 1 до 4.
Выбор [1]: 1
Плотность живых клеток, % (0...100) [50]: 150
  Введите целое число от 0 до 100.
Плотность живых клеток, % (0...100) [50]: 30
Seed (пусто = случайный): abc
  Введите целое число или оставьте пустым.
Seed (пусто = случайный): 7

Печатать каждое k-е поколение, k [1]: x
  Введите целое число от 1 до 100000.
Печатать каждое k-е поколение, k [1]: 3
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 2
  Введите целое число от 0 до 1.
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 1
```

Рис. 3: Обработка некорректного ввода (фрагмент вывода программы)

При выборе случайных начальных условий пользователь задает плотность живых клеток в процентах и `seed` генератора псевдослучайных чисел; при пустом вводе `seed` генерируется автоматически и выводится на экран, чтобы запуск можно было повторить. Затем задаются частота вывода поколений и необходимость остановки при обнаружении цикла, после чего выводится начальное поколение. На рис. 4 показано продолжение запуска с рис. 2: поле 20×20, плотность $30\,\%$, seed $= 1$; в начальном поколении 118 живых клеток (ожидаемое значение $0{,}3 \cdot 400 = 120$).

```
Плотность живых клеток, % (0...100) [50]: 30
Seed (пусто = случайный): 1

Печатать каждое k-е поколение, k [1]: 1
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 1

Поколение 0 | живых: 118
    +     +             +   +     +   +
    +     +       +   + + + +     +   +
  +   +     +     +
        + +           + +   +       +
+   +           +     +     +   +   + +
  +         +         +     +       +
    +         + +         +       +
        +           +   + +       +   +
                    + +         +
+   + +       +       + +           +
+ +           + +   +   + +   + +     +
+         +             +     +       +
            +   + + + +         + +   +
      +   +   +       +     +   +
  +     +         + + + +   +       +
+ +         +     +     + +         +
  +           + +           + +   +   +
  + +                 +             + +
                                      +
    +                 +
```

Рис. 4: Случайное заполнение поля (продолжение рис. 2)

При ручном вводе пользователь вводит строки поля сверху вниз: символы `+` и `1` обозначают живую клетку, остальные — мертвую; пустая строка завершает ввод, недостающие строки и столбцы считаются мертвыми (рис. 5). В примере на поле 9×9 введен паттерн «крест».

```
Выбор [1]: 2
Вводите строки поля сверху вниз (до 9 строк).
Живая клетка: + или 1, мёртвая: . или 0. Пустая строка - закончить ввод.
  0> .........
  1> .........
  2> .........
  3> ....+....
  4> ...+++...
  5> ....+....
  6>

Печатать каждое k-е поколение, k [1]: 1
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 0

Поколение 0 | живых: 5



       +
     + + +
       +
```

Рис. 5: Ручной ввод начальной конфигурации

На рис. 6 показано продолжение этого запуска: вывод поколений 1–3 и итог работы. Видно, что паттерн смещается вправо, как описано в разделе 4 (рис. 14), и остается в строках, в которых был задан, что соответствует свойству 5 из раздела 4. За 3 итерации цикл не найден, поэтому итог содержит соответствующее сообщение.

```
Поколение 1 | живых: 6

        +
      +   + +
        + +


Поколение 2 | живых: 6

        + + + +
        +   +


Поколение 3 | живых: 6

        +     + +
        +   + +


Итог
Выполнено итераций: 3
Живых клеток: 6 из 81
Цикл за 3 итераций не обнаружен.
```

Рис. 6: Вывод поколений и итог работы (продолжение рис. 5)

При обнаружении повторения конфигурации программа выводит поколение, на котором найден цикл, с предпериодом и периодом в заголовке, а затем итог работы (рис. 7). В примере одиночная клетка в центре поля 7×7 за 7 шагов обходит тор и возвращается в исходное положение: предпериод равен 0, период — 7. Чтобы не выводить промежуточные поколения, задано $k = 100$; поколение, на котором найден цикл, выводится всегда.

```
Двумерный клеточный автомат

Ширина поля (3...100) [20]: 7
Количество итераций (1...100000) [10]: 100

Начальные условия:
  1 - случайно
  2 - вручную
  3 - одна клетка в центре
  4 - пустое поле
Выбор [1]: 3

Печатать каждое k-е поколение, k [1]: 100
Остановиться, когда найден цикл? (1 - да, 0 - нет) [1]: 1

Поколение 0 | живых: 1



      +



Поколение 7 | живых: 1 | цикл: предпериод = 0, период = 7



      +



Итог
Выполнено итераций: 7
Живых клеток: 1 из 49
Цикл найден на поколении 7
Предпериод: 0
Период: 7
```

Рис. 7: Обнаружение цикла и итог работы

Если цикл за заданное число итераций не найден, программа сообщает об этом (рис. 8). Запуск выполнен на поле 20×20 со случайными начальными условиями (плотность $30\,\%$, seed $= 1$), $100\,000$ итераций и $k = 100\,000$, поэтому выведены только начальное (на рисунке не показано) и последнее поколения.

```
Поколение 100000 | живых: 68

  +   + +       +       +

+   +             + + + +         + + +



    +       + +         +   + + + + + +



+   +     + + + + +   +   + + + +   + +

+ +   +   +   + +     + +     + + + +

+   + +   +     + +   +       + + +   +



  +     +   +     + +     +       +

Итог
Выполнено итераций: 100000
Живых клеток: 68 из 400
Цикл за 100000 итераций не обнаружен.
```

Рис. 8: Итог работы, цикл не обнаружен (фрагмент: последнее поколение и итог)

4 Анализ клеточного автомата

Начальные условия всех экспериментов можно воспроизвести в программе: случайные начальные условия задаются плотностью и значением `seed`, простые паттерны — ручным вводом.

4.1 Свойства правила

Из таблицы истинности (табл. 1) следуют свойства, которые определяют поведение автомата.

1. Пустое поле неподвижно. Конфигурация $idx = 0$ переходит в 0, поэтому из пустого поля ничего не возникает: $t_0 = 0$, $p = 1$.
2. Полностью заполненное поле вымирает за один шаг. Конфигурация $idx = 31$ переходит в 0, поэтому на следующем шаге все клетки мертвые, далее поле пустое: $t_0 = 1$, $p = 1$.
3. Правило несимметрично относительно поворотов и отражений. Мертвая клетка рождается только при $idx \in \{2, 6, 7, 10, 15\}$, и во всех этих случаях жив западный сосед ($W = 1$). Значит, рождение возможно только при живом западном соседе: один западный сосед вызывает рождение ($idx = 2 \to 1$), тогда как одиночные северный, восточный или южный соседи — нет ($idx = 8, 4, 1 \to 0$). В результате у динамики есть выделенное направление — на восток (вправо).
4. Если живые клетки находятся только в одной строке, то клетки соседних строк видят лишь одного живого соседа по вертикали ($idx = 1$ или $idx = 8$) и не рождаются. Значит, конфигурация, занимающая одну строку, остается в этой строке. Внутри строки ($N = S = 0$) правило сводится к одномерному элементарному клеточному автомату с окрестностью (W, C, E):

$$C^{t+1} = W^t \oplus (C^t \wedge E^t),$$

   что соответствует элементарному правилу 120 по нумерации Вольфрама (зеркальное отражение правила 106).
5. Пустые строки остаются пустыми. Западный сосед клетки находится в той же строке, поэтому по свойству 3 в строке без живых клеток

ни одна клетка родиться не может. Следовательно, множество строк, содержащих живые клетки, со временем не расширяется, а может только сокращаться: паттерн растет только по горизонтали. Строки при этом не независимы: через соседей N и S они влияют на выживание и рождение клеток друг друга (например, $idx = 10$ и $idx = 15$).

4.2 Простые паттерны

Одиночная клетка

Одиночная живая клетка умирает ($idx = 16 \to 0$), а клетка справа от нее рождается, так как видит живого западного соседа ($idx = 2 \to 1$). Остальные соседи живой клетки не рождаются. В результате клетка за каждый шаг сдвигается на одну позицию вправо (рис. 9), т. е. является простейшим «глайдером», движущимся с максимально возможной для окрестности фон Неймана скоростью. Дойдя до правого края, клетка появляется у левого (тор), поэтому на поле $n \times n$ она возвращается в исходное положение через n шагов: $t_0 = 0$, $p = n$. На рис. 9 показано поле 7×7, на котором период равен 7.



Рис. 9: Движение одиночной клетки на поле 7×7

Диагонали

Диагональ из трех клеток ведет себя по-разному в зависимости от направления. У диагонали «\» (сверху слева вниз вправо) за один шаг погибают все три клетки, но рождается одна клетка справа от нижней, так

что паттерн сводится к одиночной клетке-глайдеру (рис. 10): $t_0 = 1$, $p = n$.

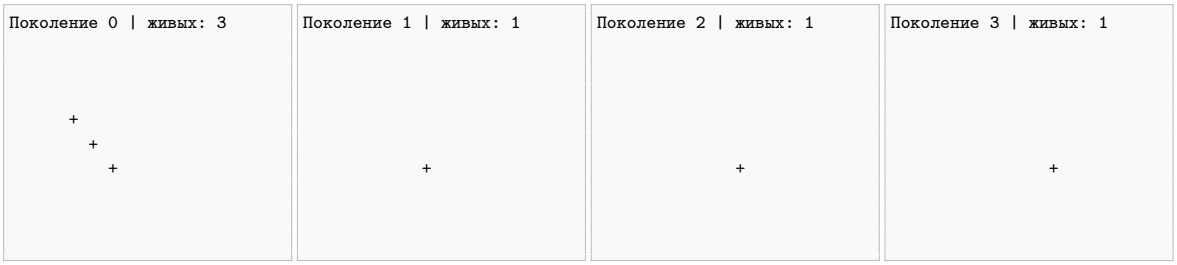


Рис. 10: Эволюция диагонали «\» на поле 9×9

Диагональ «/» (сверху справа вниз влево) сама является глайдером: за каждый шаг она целиком сдвигается на одну клетку вправо, сохраняя форму (рис. 11), поэтому $t_0 = 0$, $p = n$.

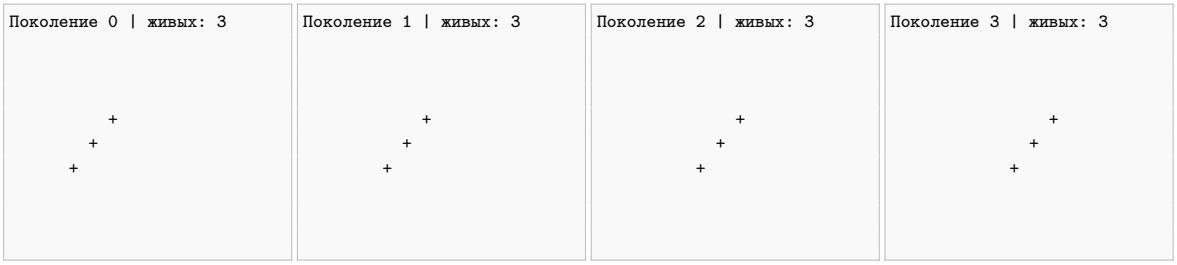


Рис. 11: Эволюция диагонали «/» на поле 9×9

Горизонтальные паттерны

Паттерн из живых клеток одной строки, согласно свойству 4, развивается только внутри этой строки по одномерному правилу 120. На рис. 12 показана эволюция пары соседних клеток: в строке возникает нерегулярная последовательность живых клеток, которая смещается вправо. На поле 20×20 такая эволюция выходит на цикл с периодом 5240 после предпериода 785 шагов (табл. 3).

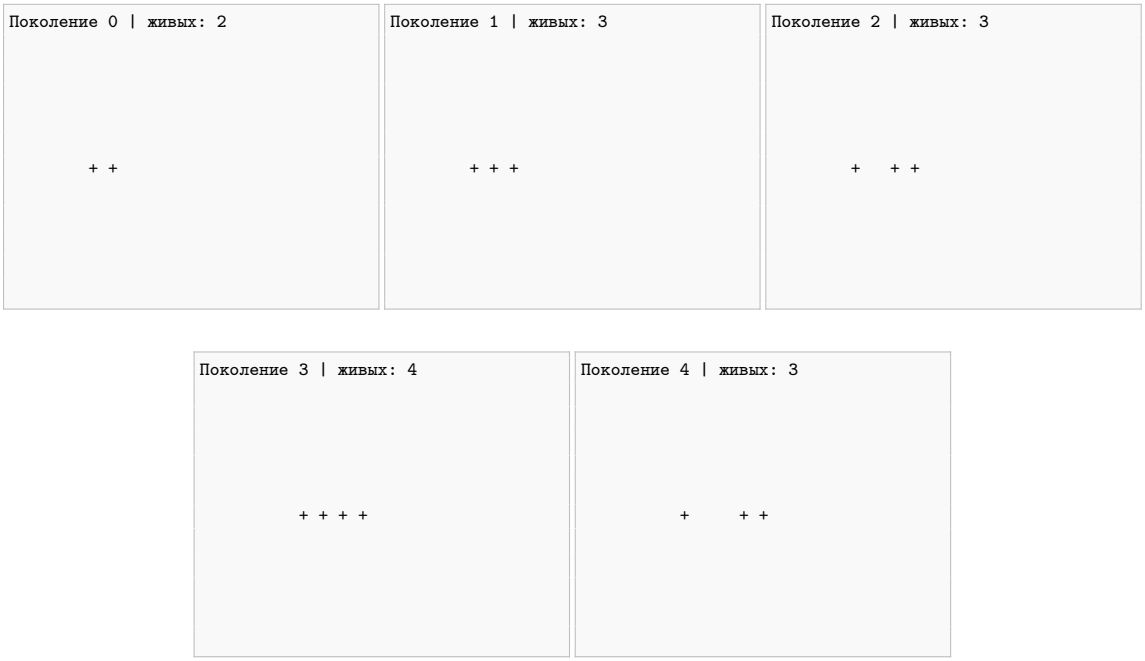


Рис. 12: Эволюция горизонтальной пары клеток на поле 11×11

Двумерные паттерны

Если живые клетки находятся в нескольких строках, то строки влияют друг на друга через северного и южного соседей, и эволюция уже не сводится к одномерному правилу. Однако по свойству 5 паттерн никогда не выходит за пределы строк, в которых он был задан. На рис. 13 и 14 показаны первые шаги эволюции вертикальной пары клеток и «креста»: в обоих случаях паттерн разрастается по горизонтали внутри своих строк и одновременно смещается вправо.

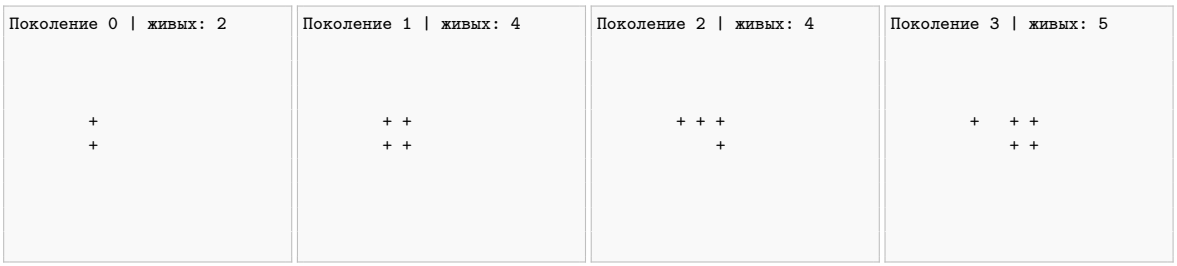


Рис. 13: Эволюция вертикальной пары клеток на поле 9×9

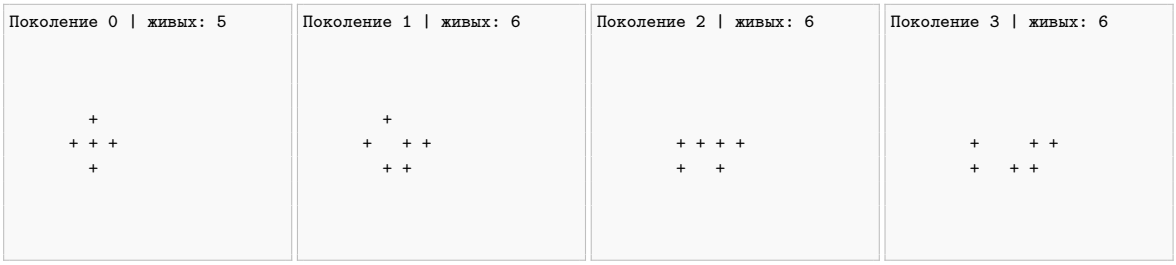


Рис. 14: Эволюция паттерна «крест» на поле 9×9

Вертикальная пара за один шаг превращается в квадрат 2×2 (рис. 13, поколение 1), поэтому дальше эти паттерны эволюционируют одинаково, а их предпериоды в таблице 3 отличаются на 1. Сам квадрат 2×2 показан на рис. 15: верхняя строка вытягивается вправо, нижняя теряет левые клетки.

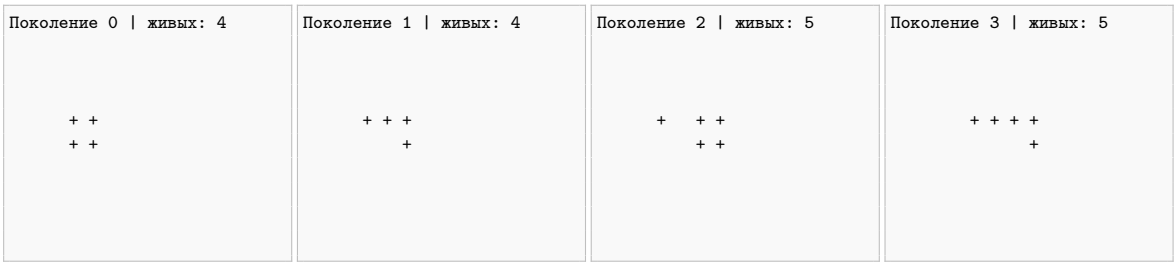


Рис. 15: Эволюция квадрата 2×2 на поле 9×9

На рис. 16–18 показаны паттерны, занимающие три строки: буква «L», вертикальная линия и квадрат 3×3. Средняя строка «L» с одной клеткой сразу погибает и по свойству 5 остается пустой, поэтому верхняя и нижняя строки дальше развиваются независимо, каждая по одномерному правилу 120 (свойство 4); этим объясняется совпадение периода «L» с периодом горизонтальных паттернов (табл. 3). У вертикальной линии и квадрата 3×3 все три строки сохраняют живые клетки, и паттерн растет вправо, оставаясь в своих трех строках.

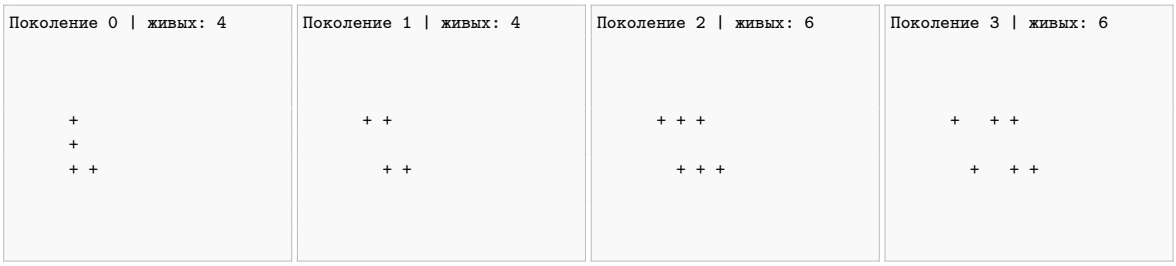


Рис. 16: Эволюция паттерна «L» на поле 9×9

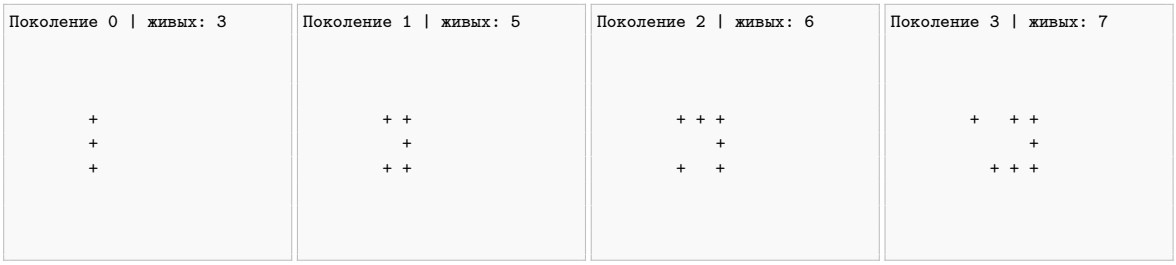


Рис. 17: Эволюция вертикальной линии из 3 клеток на поле 9×9

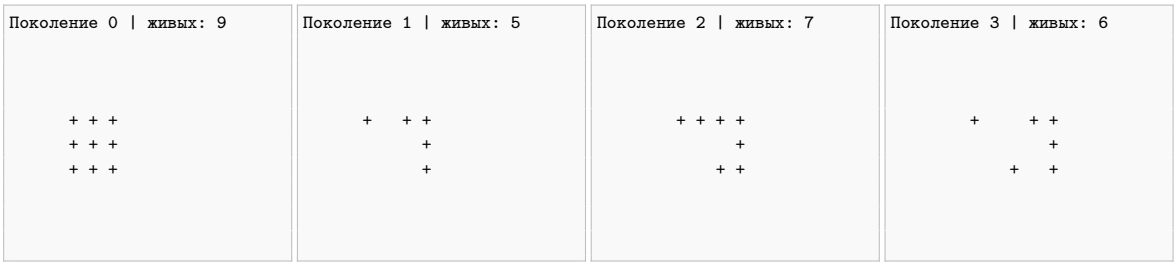


Рис. 18: Эволюция квадрата 3×3 на поле 9×9

Результаты запуска автомата на простых паттернах, помещенных в центр поля 20×20, приведены в таблице 3.

Таблица 3: Простые паттерны на поле 20×20

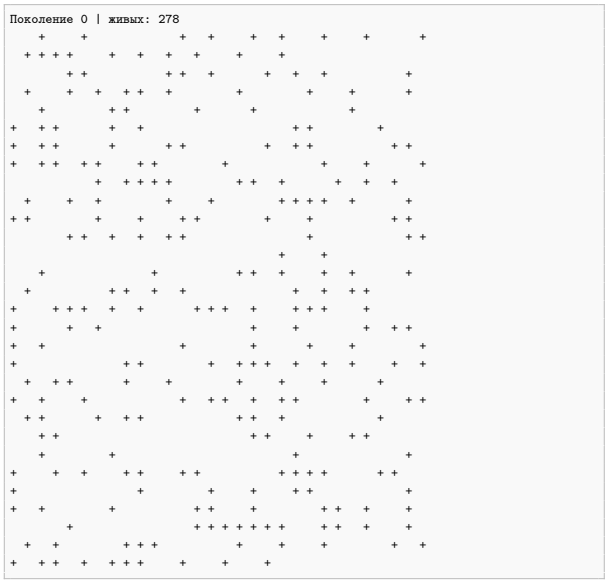
Паттерн	Предпериод t_0	Период p	Живых*
одна клетка	0	20	1
диагональ из 3 клеток «\»	1	20	1
диагональ из 3 клеток «/»	0	20	3
крест	11	40	9
две клетки по горизонтали	785	5240	10
горизонтальная линия из 3 клеток	784	5240	10
две клетки по вертикали	428	21240	13
квадрат 2×2	427	21240	13
квадрат 3×3	1036	5240	21
буква «L»	786	5240	20
вертикальная линия из 3 клеток	не найден за 10^5 шагов		15

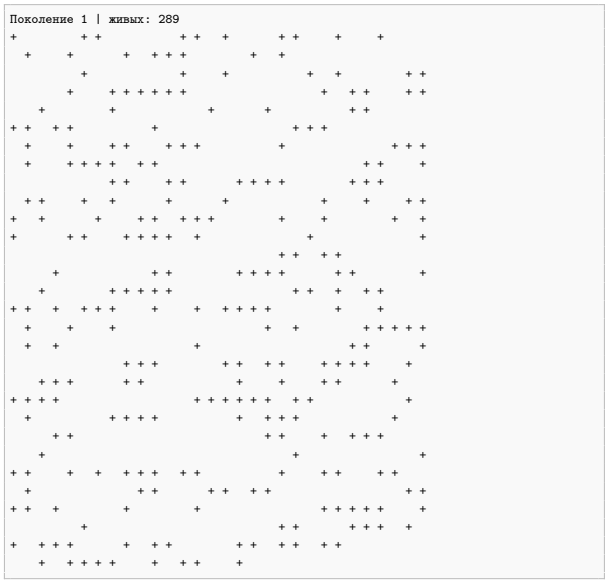
Видно, что даже паттерны из 2–5 клеток в большинстве случаев проходят длинный переходный процесс (сотни шагов) и выходят на циклы длиной в тысячи шагов. Исключения — паттерны, которые за несколько

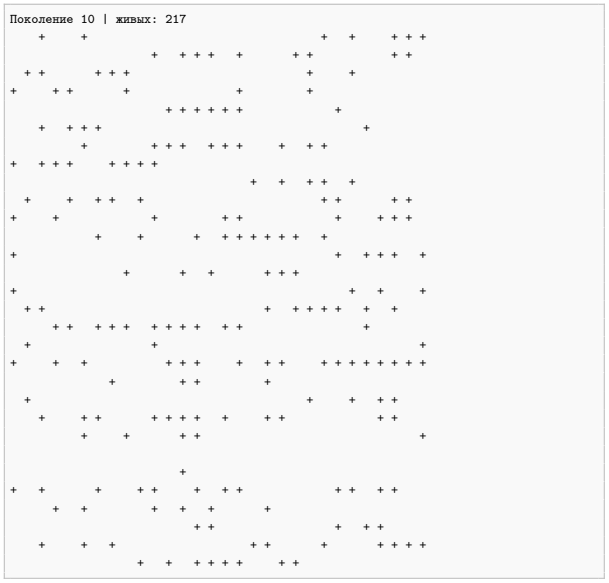
шагов сводятся к одиночной движущейся клетке (диагональ «\») или к небольшой движущейся структуре (крест), а также диагональ «/», которая сама является глайдером.

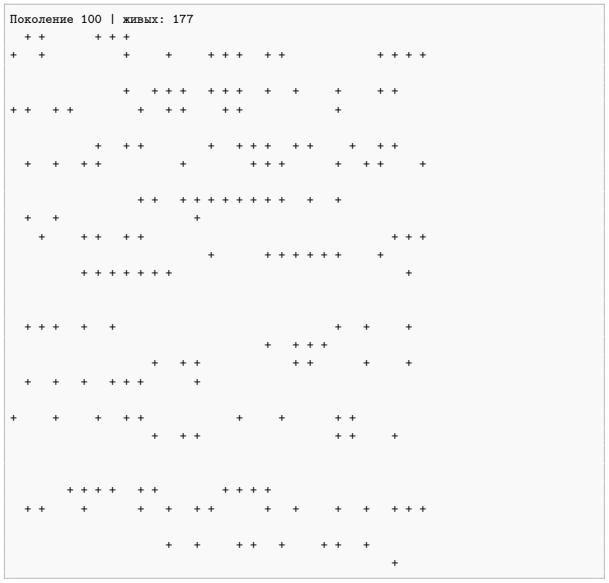
4.3 Случайные начальные условия

На рис. 19 показана эволюция случайной начальной конфигурации на поле 30×30 с плотностью живых клеток 30 % (`seed` = 1). Уже через несколько шагов поле перестает напоминать начальную конфигурацию, и далее картина остается неупорядоченной: устойчивых или периодических локальных структур не возникает, характерны короткие горизонтальные цепочки живых клеток, движущиеся вправо.









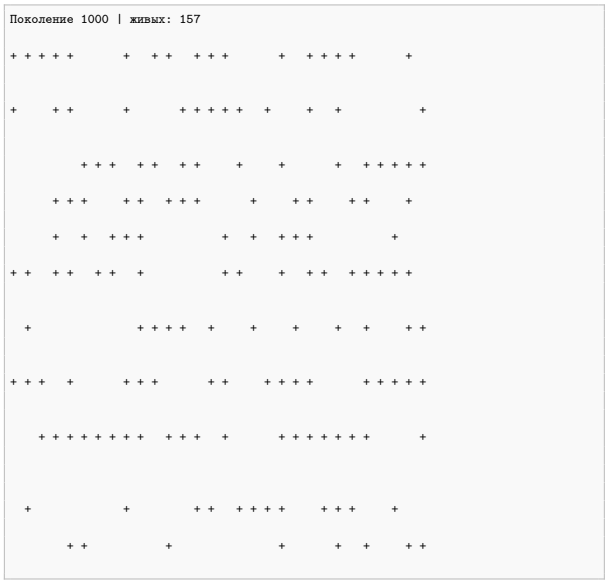


Рис. 19: Эволюция случайной конфигурации на поле 30×30 (плотность $30\,\%$, seed $= 1$)

Для оценки того, как меняется число живых клеток, были проведены эксперименты на поле 50×50: для каждой начальной плотности (10, 30, 50, 70 и 90 %) выполнено 20 запусков (seed = 1..20) по 100 поколений, затем доля живых клеток усреднена по запускам (рис. 20). Независимо от начальной плотности доля живых клеток быстро (за 5–10 поколений) смещается к значениям около 0,1–0,25: при плотностях 50, 70 и 90 % она резко падает, при 10 % немного возрастает. К 100-му поколению доли для плотностей 30, 50 и 70 % сближаются (0,204; 0,208; 0,202), а для плотностей 10 и 90 % продолжают медленно сходиться к тому же уровню (0,161 и 0,149).

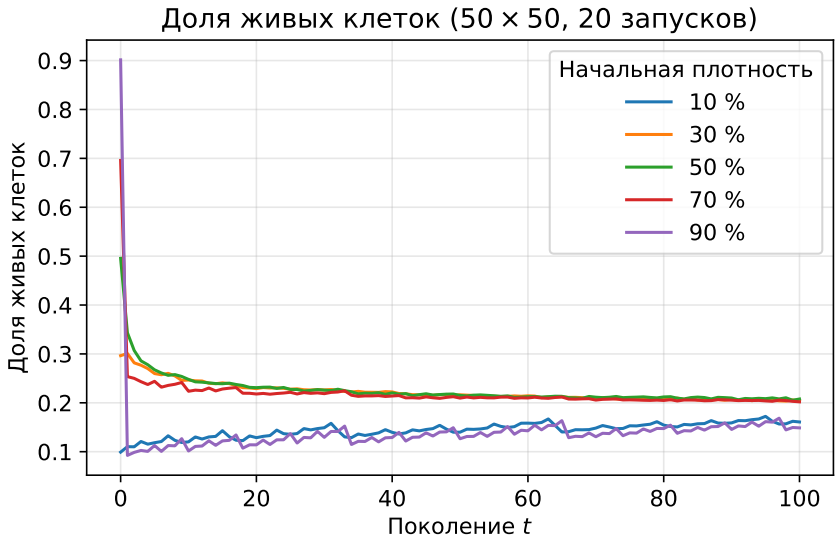


Рис. 20: Доля живых клеток в зависимости от номера поколения (поле 50×50, среднее по 20 запускам)

4.4 Периодичность

В таблице 4 приведены предпериоды и периоды, найденные программой для полей $n \times n$ при $n = 3..16$, плотности 50 % и `seed` = 1..10 (ограничение — 10^5 шагов, как в программе).

Таблица 4: Предпериоды и периоды для случайных конфигураций (плотность 50 %, 10 запусков)

n	2^{n^2}	t_0 (мин–макс)	Найденные периоды p	Не найден
3	$5{,}1 \cdot 10^2$	0–5	1, 3	0
4	$6{,}6 \cdot 10^4$	1–14	2, 4, 18	0
5	$3{,}4 \cdot 10^7$	1–12	5, 15, 45	0
6	$6{,}9 \cdot 10^{10}$	8–24	1, 3, 6	0
7	$5{,}6 \cdot 10^{14}$	2–46	7, 28, 49, 196	0
8	$1{,}8 \cdot 10^{19}$	10–57	30, 36, 120, 180, 360	0
9	$2{,}4 \cdot 10^{24}$	30–154	9, 36, 54, 108	0
10	$1{,}3 \cdot 10^{30}$	30–118	205, 410, 1640	0
11	$2{,}7 \cdot 10^{36}$	47–139	176, 1232	0
12	$2{,}2 \cdot 10^{43}$	50–311	1, 12, 24, 168	0
13	$7{,}5 \cdot 10^{50}$	125–190	416, 4576, 12896, 22880	5
14	$1{,}0 \cdot 10^{59}$	179–427	15680, 28224	2
15	$5{,}4 \cdot 10^{67}$	134–289	450, 32850, 59850	7
16	$1{,}2 \cdot 10^{77}$	189–745	2688, 13440	0

На полях $n = 17, 18$ циклы еще находятся (например, $t_0 = 746$, $p = 2533$ при $n = 17$; $t_0 = 554$, $p = 23\,976$ при $n = 18$), но уже при $n = 20$ и $n = 25$ ни в одном из запусков цикл за 10^5 шагов не обнаружен; при $n = 30$ (плотность 30 %) цикл не найден за 10^5 шагов.

Из таблицы видно, что

- при $n \geq 10$ предпериод невелик по сравнению с периодом, на малых полях, наоборот, переходный процесс может быть длиннее короткого цикла (например, при $n = 6$: $t_0 = 8..24$, $p = 1..6$);
- период в целом быстро растет с размером поля (от единиц и десятков при $n \leq 6$ до десятков тысяч при $n \geq 13$), хотя рост неравномерен и зависит от арифметических свойств n;
- периоды часто кратны n, что связано со сдвигом структур вправо: конфигурация возвращается в исходное положение после целого числа оборотов вокруг тора;
- найденные периоды несравнимо меньше числа 2^{n^2} возможных конфигураций, однако при $n \geq 20$ они уже недоступны для прямого

перебора.

4.5 Классификация по Вольфраму

По результатам анализа автомат с правилом 20 874 436 (порядок $C\,N\,E\,W\,S$) относится к классу III (хаотическое поведение):

- эволюция случайных конфигураций не приводит ни к однородному состоянию (класс I), ни к набору простых устойчивых или периодических структур (класс II): поле остается неупорядоченным;
- длины циклов на торе быстро растут с размером поля и уже при $n \geq 20$ не наблюдаются за 10^5 шагов;
- даже простые паттерны из нескольких клеток порождают длительную нерегулярную эволюцию.

У автомата есть и черты других классов: на малых полях ($n \leq 6$) встречаются неподвижные точки и короткие циклы, а одиночная клетка и диагональ из трех клеток «/» являются движущимися локализованными структурами («глайдерами»). Однако на случайных начальных конфигурациях такие структуры не сохраняются и растворяются в хаотическом фоне, поэтому оснований для отнесения автомата к классу IV нет.

Заключение

В ходе выполнения лабораторной работы был реализован двумерный клеточный автомат с окрестностью фон Неймана, тороидальными граничными условиями и правилом перехода 20 874 436. Программа позволяет задавать ширину поля и количество итераций, выбирать начальные условия (случайные с заданной плотностью и `seed`, ручной ввод, одна клетка в центре или пустое поле), а также автоматически обнаруживает цикл и вычисляет его предпериод и период.

Анализ показал, что рождение клетки возможно только при живом западном соседе, поэтому у динамики есть выделенное направление — структуры смещаются вправо, а одиночная клетка и диагональ «/» из трех клеток являются глайдерами с периодом n на торе. Конфигурации, занимающие одну строку, эволюционируют по одномерному элементарному правилу 120. Строки без живых клеток остаются пустыми, поэтому паттерны растут только по горизонтали. Случайные конфигурации остаются неупорядоченными. Длины циклов быстро растут с размером поля: при $n \leq 18$ циклы находятся и их периоды достигают десятков тысяч шагов, а при $n \geq 20$ за 10^5 шагов цикл не обнаружен ни в одном запуске. По классификации Вольфрама автомат отнесен к классу III (хаотическое поведение).

Достоинства реализации

- Синхронное обновление реализовано обменом ссылок на два массива без копирования, сложность шага — $O(n^2)$.
- Ввод устойчив к ошибкам: некорректные значения и выход за диапазон приводят к повторному запросу, а при окончании ввода используются значения по умолчанию.
- Случайные начальные условия воспроизводимы благодаря `seed`, который выводится, даже если был сгенерирован автоматически.

Недостатки реализации

- Номер правила и порядок клеток в индексе заданы в коде, изменить их при работе программы нельзя.
- Память растёт с числом шагов.
- Поле выводится символами в консоль, поэтому большие поля ($n > 60$) неудобно анализировать.

Масштабирование

- Графический интерфейс.
- Параллельное вычисление шага для больших полей.

Список литературы

[1] Секция «Телематика». https://tema.spbstu.ru/algorithm/. (Дата обращения: 24.10.2026).

[2] Нейман Дж. фон. Теория самовоспроизводящихся автоматов / пер. с англ. В. Л. Стефанюка. — М.: Мир, 1971. — 381 с.

[3] Wolfram S. Universality and complexity in cellular automata // Physica D: Nonlinear Phenomena. — 1984. — Vol. 10, no. 1–2. — P. 1–35.

Приложение. Исходный код программы

Класс CellularAutomaton

```
package ru.spbstu.khramov;

import java.util.BitSet;
import java.util.HashMap;
import java.util.Map;
import java.util.Random;

public class CellularAutomaton {
    private final long rule;
    private boolean[][] cells;
    private final int size;
    private boolean[][] next;

    private int generation;

    private final Map<BitSet, Integer> history = new HashMap<>();
    private boolean cycleFound;
    private int transientLength;
    private int period;

    public CellularAutomaton(int numberOption, int constValue, int year,
        int month, int day, int size) {
        this.rule = (long) numberOption * constValue * year * month * day;
        this.size = size;
        this.cells = new boolean[this.size][this.size];
        this.next = new boolean[this.size][this.size];
    }

    int newState(int idx) {
        return bit(rule, idx);
    }

    static int bit(long x, int k) {
        long shifted = x >> k;
        long lastBit = shifted & 1;
        return (int) lastBit;
    }

    public boolean getCell(int row, int col) {
        return cells[row][col];
    }

    public void setCell(int row, int col, boolean value) {
```

```
43          cells[row][col] = value;
44      }
45
46      private int wrap(int coord) {
47          return (coord % size + size) % size;
48      }
49
50      private boolean getWrapped(int row, int col) {
51          return cells[wrap(row)][wrap(col)];
52      }
53
54      private static int toBit(boolean value) {
55          return value ? 1 : 0;
56      }
57
58      int neighborhoodIndex(int row, int col) {
59          int n = toBit(getWrapped(row - 1, col));
60          int c = toBit(getWrapped(row, col));
61          int s = toBit(getWrapped(row + 1, col));
62          int w = toBit(getWrapped(row, col - 1));
63          int e = toBit(getWrapped(row, col + 1));
64          return c * 16 + n * 8 + e * 4 + w * 2 + s;
65      }
66
67      public int getGeneration() {
68          return generation;
69      }
70
71      public int getSize() {
72          return size;
73      }
74
75      public void step() {
76          if (history.isEmpty()) {
77              history.put(snapshot(), generation);
78          }
79
80          for (int i = 0; i < size; i++) {
81              for (int j = 0; j < size; j++) {
82                  int idx = neighborhoodIndex(i, j);
83                  next[i][j] = newState(idx) == 1;
84              }
85          }
86
87          boolean[][] tmp = cells;
88          cells = next;
89          next = tmp;
90
```

```
91          generation++;
92          checkCycle();
93      }
94
95      private void checkCycle() {
96          if (cycleFound) {
97              return;
98          }
99          Integer prev = history.putIfAbsent(snapshot(), generation);
100         if (prev != null) {
101             cycleFound = true;
102             transientLength = prev;
103             period = generation - prev;
104         }
105     }
106
107     private BitSet snapshot() {
108         BitSet bits = new BitSet(size * size);
109         for (int row = 0; row < size; row++) {
110             for (int col = 0; col < size; col++) {
111                 if (cells[row][col]) {
112                     bits.set(row * size + col);
113                 }
114             }
115         }
116         return bits;
117     }
118
119     public boolean isCycleFound() {
120         return cycleFound;
121     }
122
123     public int getTransientLength() {
124         return transientLength;
125     }
126
127     public int getPeriod() {
128         return period;
129     }
130
131     public int population() {
132         int count = 0;
133         for (int row = 0; row < size; row++) {
134             for (int col = 0; col < size; col++) {
135                 if (cells[row][col]) {
136                     count++;
137                 }
138             }
```

```
        }
        return count;
    }

    public void clear() {
        for (int row = 0; row < size; row++) {
            for (int col = 0; col < size; col++) {
                cells[row][col] = false;
            }
        }
    }

    public void randomize(double density, long seed) {
        Random random = new Random(seed);
        for (int row = 0; row < size; row++) {
            for (int col = 0; col < size; col++) {
                cells[row][col] = random.nextDouble() < density;
            }
        }
    }

}
```

Листинг 30: CellularAutomaton.java

Класс Main

```
package ru.spbstu.khramov;

import static java.lang.IO.*;

public class Main {
    private Main() {
    }

    static void main() {
        println("Двумерный клеточный автомат");
        println();

        int size = readInt("Ширина поля (3...100)", 3, 100, 20);
        int iterations = readInt("Количество итераций (1...100000)", 1,
            100_000, 10);

        CellularAutomaton automaton = new CellularAutomaton(43, 11, 2006,
            11, 2, size);
        initField(automaton);

```

```
19          println();
20          int printEvery = readInt("Печатать каждое k-е поколение, k", 1,
                100_000, 1);
21          boolean stopOnCycle = readInt("Остановиться, когда найден цикл? (1
                - да, 0 - нет)", 0, 1, 1) == 1;
22
23          println();
24          printGeneration(automaton);
25          for (int i = 1; i <= iterations; i++) {
26              boolean hadCycle = automaton.isCycleFound();
27              automaton.step();
28              boolean cycleJustFound = !hadCycle && automaton.isCycleFound()
                    ;
29
30              if (i % printEvery == 0 || i == iterations || cycleJustFound)
                    {
31                  printGeneration(automaton);
32              }
33              if (cycleJustFound && stopOnCycle) {
34                  break;
35              }
36          }
37
38          printSummary(automaton);
39      }
40
41      static void printGeneration(CellularAutomaton automaton) {
42          String header = "Поколение " + automaton.getGeneration() + " | жив
                ых: " + automaton.population();
43          if (automaton.isCycleFound()) {
44              header += " | цикл: предпериод = " + automaton.
                    getTransientLength() + ", период = " + automaton.getPeriod
                    ();
45          }
46          println(header);
47          printField(automaton);
48      }
49
50      static void printSummary(CellularAutomaton automaton) {
51          int size = automaton.getSize();
52          println("Итог");
53          println("Выполнено итераций: " + automaton.getGeneration());
54          println("Живых клеток: " + automaton.population() + " из " + size
                * size);
55          if (automaton.isCycleFound()) {
56              int t = automaton.getTransientLength();
57              int p = automaton.getPeriod();
58              println("Цикл найден на поколении " + (t + p));
```

```
59              println("Предпериод: " + t);
60              println("Период: " + p + (p == 1 ? " (неподвижная точка)" : ""
                    ));
61          } else {
62              println("Цикл за " + automaton.getGeneration() + " итераций не
                    обнаружен.");
63          }
64      }
65
66      static void initField(CellularAutomaton automaton) {
67          println();
68          println("Начальные условия:");
69          println("  1 - случайно");
70          println("  2 - вручную");
71          println("  3 - одна клетка в центре");
72          println("  4 - пустое поле");
73          int mode = readInt("Выбор", 1, 4, 1);
74
75          switch (mode) {
76              case 1 -> {
77                  int percent = readInt("Плотность живых клеток, % (0...100)
                        ", 0, 100, 50);
78                  long seed = readSeed();
79                  automaton.randomize(percent / 100.0, seed);
80              }
81              case 2 -> readManual(automaton);
82              case 3 -> {
83                  int center = automaton.getSize() / 2;
84                  automaton.setCell(center, center, true);
85              }
86              default -> automaton.clear();
87          }
88      }
89
90      static long readSeed() {
91          while (true) {
92              String s = readLine("Seed (пусто = случайный): ");
93              if (s.isEmpty()) {
94                  long seed = System.nanoTime();
95                  println("Используется seed = " + seed);
96                  return seed;
97              }
98              try {
99                  return Long.parseLong(s);
100             } catch (NumberFormatException e) {
101                 println("  Введите целое число или оставьте пустым.");
102             }
103         }
```

```
104     }
105
106     static void readManual(CellularAutomaton automaton) {
107         int size = automaton.getSize();
108         println("Вводите строки поля сверху вниз (до " + size + " строк).");
109         println("Живая клетка: + или 1, мёртвая: . или 0. Пустая строка - закончить ввод.");
110         automaton.clear();
111         for (int row = 0; row < size; row++) {
112             String line = readln(String.format("%3d> ", row));
113             if (line == null || line.isBlank()) {
114                 break;
115             }
116             for (int col = 0; col < Math.min(line.length(), size); col++) {
117                 char ch = line.charAt(col);
118                 automaton.setCell(row, col, ch == '+' || ch == '1');
119             }
120         }
121     }
122
123     static int readInt(String prompt, int min, int max, int def) {
124         while (true) {
125             String s = readLine(prompt + " [" + def + "]: ");
126             if (s.isEmpty()) {
127                 return def;
128             }
129             try {
130                 int value = Integer.parseInt(s);
131                 if (value >= min && value <= max) {
132                     return value;
133                 }
134             } catch (NumberFormatException ignored) {
135             }
136             println("  Введите целое число от " + min + " до " + max + ".");
137         }
138     }
139
140     static String readLine(String prompt) {
141         String s = readln(prompt);
142         return s == null ? "" : s.trim();
143     }
144
145     static void printField(CellularAutomaton automaton) {
146         for (int row = 0; row < automaton.getSize(); row++) {
147             for (int col = 0; col < automaton.getSize(); col++) {
```

```
148                 print(automaton.getCell(row, col) ? "+ " : "  ");
149             }
150             println();
151         }
152         println();
153     }
154 
155 }
```

Листинг 31: Main.java